

Decisiones Arquitectónicas (ADRs)

Polizing — Sistema de Gestión de Pólizas de Seguros

Entregable 4 · Práctica de Arquitectura y Diseño de Sistemas 2026

Grupo 6 — Felder, Ducos, Ferraro, Fueyo, Kravetz, Echavarría

Índice

- [ADR-001](#) — Arquitectura del Backend
- [ADR-002](#) — Persistencia de Datos
- [ADR-003](#) — Comunicación entre Componentes
- [ADR-004](#) — Infraestructura y Despliegue
- [ADR-005](#) — Selección del proveedor de mensajería

ADR-001: Arquitectura del Backend

ID	Título	Estado	Fecha
ADR-001	Estructura y límites de los servicios del backend	Aceptada	01/06/2026

1. Contexto

Polizing requiere dos interfaces de usuario con ciclos de vida y necesidades de escalado diferentes:

- Panel web: productoras de seguros consultan/editan pólizas, clientes, siniestros. Tráfico aleatorio, acceso puntual durante horario laboral.
- Chatbot WhatsApp: clientes finales consultan estado de pólizas 24/7. Tráfico predecible pero sostenido.

Motivadores que obligan a tomar una decisión arquitectónica:

- ❶ Ciclos de deployment independientes: el chatbot evoluciona a ritmo diferente que la web (actualizaciones de prompts, intención de NLU, templates de WhatsApp). Cada cambio en un servicio no debe bloquear al otro.
- ❷ Escalado heterogéneo: la web (Next.js sobre Vercel serverless) escala a cero. El chatbot (FastAPI containerizado) necesita estar siempre disponible 24/7 con costos predecibles.
- ❸ Restricciones de runtime: la integración con WhatsApp Cloud API requiere un webhook HTTPS estable, sesiones largas y estado conversacional en BD. El stack serverless Next.js no encaja con ese perfil; FastAPI sobre proceso persistente no encaja con un panel React server-rendered.
- ❹ Datos compartidos en tiempo real: ambos servicios consultan/escriben datos de negocio comunes (pólizas, clientes, siniestros). Desincronización = inconsistencia crítica.

2. Alternativas consideradas

Alternativa	Ventaja principal en este contexto	Desventaja / Motivo de descarte
Monolito único (Next.js)	Un solo deploy, un solo lenguaje (TypeScript), un ORM, una BD. Complejidad operacional mínima.	Requeriría embeber lógica de chatbot en Node.js (ineficiente para prompts NLP). No escala independientemente. El chatbot quedaría sujeto al ciclo de deploy de la web.
Dos servicios independientes con BD separadas	Ciclos de deploy independientes. Escalado heterogéneo sin restricciones. Sin colisión de esquemas.	Cada servicio con su propia BD → desincronización de datos core. Para mantener consistencia se necesitaría ETL o API de sincronización, agregando latencia y complejidad.
Microservicios con BD compartida y API HTTP (ELEGIDA)	Ciclos de deploy independientes. Datos sincronizados en tiempo real mediante BD relacional compartida. Comunicación HTTP	— (esta fue la elegida)

	síncrona para datos de negocio.	
--	---------------------------------	--

3. Decisión tomada

Se decide: arquitectura de dos aplicaciones independientes (web + chatbot) que comparten una única base de datos PostgreSQL, con esquemas aislados (public para Polizing-Dashboard con Prisma, chatbot para el chatbot con SQLAlchemy), y se comunican entre sí mediante REST HTTP síncrono con autenticación por API Key para el acceso a datos de negocio compartidos.

Fundamentación:

1. Ciclos de deployment independientes: cada servicio despliega en su plataforma con su propio CI/CD. El chatbot en Render, la web en Vercel. Cambios en prompts del chatbot no requieren rebuild del panel, y viceversa.
2. Escalado heterogéneo sin sobre costo: Next.js escala a serverless (costo marginal en inactividad), el chatbot mantiene instancia gratuita con shutdown automático cada 15 minutos de inactividad.
3. Datos sincronizados en tiempo real: ambos servicios acceden a la misma BD PostgreSQL. El chatbot persiste sus datos operacionales directamente en su esquema chatbot. Para datos de negocio del panel (clientes, pólizas) utiliza la API HTTP del panel, evitando queries cross-schema.

4. Consecuencias

✓ Consecuencias positivas	⚠ Trade-offs / costos
Ciclos de deploy independientes: cambios en chatbot no tocan Next.js.	Acoplamiento indirecto vía BD: cambios de esquema requieren coordinación.
Escalado optimizado por carga: web serverless + API containerizada.	Dos ORMs, dos deployment targets, dos paneles de monitoreo.
Sincronización de datos en tiempo real sin ETL manual.	API HTTP introduce latencia de red (~100ms) y dependencia temporal.

Relación con otros ADRs: depende de ADR-002 (BD compartida) y ADR-003 (comunicación HTTP), se concreta en ADR-004 (despliegue).

ADR-002: Persistencia de Datos

ID	Título	Estado	Fecha
ADR-002	Eleccion de la capa de persistencia de datos del panel	Aceptada	01/06/2026

1. Contexto

El sistema gestiona datos críticos de negocio: pólizas activas, pagos, siniestros, clientes. La integridad de estos datos es no-negociable en el dominio de seguros.

Motivadores:

- ➊ Modelo fuertemente relacional: las entidades centrales (clientes, pólizas, siniestros, pagos, aseguradoras) tienen relaciones 1:N y N:M reales. Las consultas más frecuentes requieren joins.
- ➋ Consistencia transaccional: crear un siniestro (inserta en siniestros + N documentos) o validar un pago (actualiza pago + propaga estado a pólizas) deben ser operaciones atómicas.
- ➌ Trazabilidad por actor: cada tabla de negocio incluye `modificado_por_id` y `modificado_en` para auditoría. Requiere foreign keys, índices y constraints.
- ➍ Dos ORMs en la misma BD: Prisma (Polizing-Dashboard) y SQLAlchemy (chatbot) necesitan convivir sin colisiones de nombres de tablas ni migraciones cruzadas.

2. Alternativas consideradas

Alternativa	Ventaja principal en este contexto	Desventaja / Motivo de descarte
MongoDB (NoSQL documental)	Esquema flexible, escalabilidad horizontal nativa, rápida inserción de datos.	No soporta joins nativos. Transacciones multi-documento lentas. Consultas de reportes requerirían lógica compleja en app. El modelo relacional del dominio fuerza denormalización que causa inconsistencias.
Firestore / Firebase (NoSQL cloud)	Serverless, auto-escala, sincronización en tiempo real. Simplicidad operacional.	Queries limitadas (sin joins, sin agregaciones). Costos impredecibles. Vendor lock-in severo. Sin soporte para constraints de integridad que el dominio de seguros exige.
PostgreSQL (Supabase) con esquemas aislados (ELEGIDA)	ACID completo, joins nativos, tipos avanzados, esquemas SQL para aislamiento entre ORMs. Supabase elimina la operación del motor.	— (esta fue la elegida)

3. Decisión tomada

Se decide: PostgreSQL administrado por Supabase como base de datos única del sistema, con aislamiento de esquemas SQL (public para Polizing-Dashboard/Prisma, chatbot para chatbot/SQLAlchemy).

Fundamentación:

1. Modelo relacional: PostgreSQL ejecuta nativamente todos los joins que el panel necesita (clientes con pólizas, siniestros con su póliza, pagos agrupados).
2. Consistencia transaccional: las transacciones ACID garantizan atomicidad en operaciones multi-tabla. CHECK constraints protegen invariantes a nivel motor.
3. Trazabilidad nativa: los campos `modificado_por_id` / `modificado_en` con foreign keys se expresan naturalmente. Las consultas de auditoría son queries SQL directas.
4. Migraciones con dos ORMs sin colisiones: esquemas aislados (feature nativa de PostgreSQL) resuelven el motivador 4. Prisma usa public, SQLAlchemy usa chatbot.

4. Consecuencias

✓ Consecuencias positivas	⚠ Trade-offs / costos
Consistencia ACID: datos nunca en estado inconsistente.	Escalabilidad horizontal limitada: PostgreSQL escala mejor verticalmente.
Auditoría nativa con <code>modified_por</code> y <code>modificado_en</code> .	Esquema rígido: cambios requieren migraciones explícitas.
Migraciones independientes por ORM sin colisiones.	Conexiones limitadas: pgBouncer tiene máximo. Dos ORMs consumen conexiones rápido.

Relación con otros ADRs: sostiene ADR-001. Es prerequisite de ADR-003 y ADR-004.

ADR-003: Comunicación entre Componentes

ID	Título	Estado	Fecha
ADR-003	Estrategia de comunicación e integración entre componentes internos	Aceptada	01/06/2026

1. Contexto

Como consecuencia de ADR-001, el sistema tiene dos servicios independientes (Polizing-Dashboard y chatbot) que necesitan colaborar:

- El chatbot necesita validar clientes por teléfono, consultar pólizas activas, generar tarjetas de circulación, y registrar siniestros/comprobantes en el sistema.
- El panel web necesita visibilidad de conversaciones y solicitudes que ingresan por WhatsApp.

Motivadores:

- ❶ Patrón request/response del chatbot: cada mensaje entrante exige respuesta en segundos. El chatbot necesita el dato (¿existe el cliente?, ¿qué pólizas tiene?) antes de responder.
- ❷ Datos operacionales propios del chatbot (conversaciones, mensajes, estado de flujo): son de alta frecuencia, pertenecen exclusivamente al chatbot, y no deben acoplar su estructura a la web.
- ❸ Seguridad server-to-server: la API del panel expone datos sensibles (DNIs, pólizas, comprobantes). Solo el chatbot debe consumirla. Autenticación simple y verificable.
- ❹ Volumen acotado por cadencia humana: el chatbot atiende clientes escribiendo a mano. El volumen pico está muy por debajo de lo que justificaría una capa de mensajería.

2. Alternativas consideradas

Alternativa	Ventaja principal en este contexto	Desventaja / Motivo de descarte
Consulta directa a BD compartida para todo	Latencia mínima (<1ms). Sin acoplamiento HTTP. Sin SPOF de API.	El chatbot necesitaría conocer la estructura del esquema public del panel. Consultas cross-schema complejas y alto riesgo de romper al migrar el esquema del panel.
Cola de mensajes (RabbitMQ / SQS)	Desacoplamiento temporal. Tolerancia a fallos. Buena para picos de carga.	No encaja con el motivador 1 (request/response síncrono). Suma infraestructura innecesaria para el volumen del sistema (motivador 4).
REST síncrono con API Key + acceso directo a BD propia (ELEGIDA)	HTTP con API Key para datos de negocio del panel. Acceso directo al esquema chatbot en BD compartida para datos operacionales.	— (esta fue la elegida)

3. Decisión tomada

Se decide: estrategia híbrida —

(a) el chatbot accede directamente a su propio esquema (chatbot) en la BD compartida para datos operacionales (conversaciones, mensajes, estado de flujo), y

(b) consume la API REST del panel vía HTTPS con autenticación por header X-API-Key para toda operación sobre datos de negocio (clientes, pólizas, siniestros, comprobantes de pago).

Las apis expuestas con el panel se entregan como documentación pertinente para esta entrega, y queda disponible también un archivo .yaml para ver eficazmente la documentación de APIs.

Fundamentación:

1. Resuelve el motivador 1 (request/response): el chatbot llama a la API del panel, recibe el dato y responde al cliente en el mismo turno. Sin callbacks ni estado intermedio.
2. Resuelve el motivador 2 (datos propios): al acceder directamente a su esquema en la BD compartida para conversaciones y mensajes, evita acoplar su estructura interna al panel.
3. Resuelve el motivador 3 (seguridad): el header X-API-Key se valida en un middleware de Next.js. Simple de implementar, permite rotar el secreto sin tocar usuarios finales.
4. Volumen acotado → solución simple: REST sobre HTTPS es el camino más debugueable. Cualquier integrante del grupo lo entiende sin onboarding adicional.

4. Consecuencias

✓ Consecuencias positivas	⚠ Trade-offs / costos
Latencia mínima para datos operacionales del chatbot.	Acoplamiento temporal vía HTTP: si el panel cae, el chatbot no responde sobre datos de negocio.
Contrato explícito y versionable (endpoints en <code>main_system_client.py</code>).	Sin retry automático: un timeout en POST puede crear el recurso sin confirmación.
API Key permite cortar acceso del chatbot si se compromete.	Dos formas de acceso a la BD = dos pools de conexiones que monitorear.

Relación con otros ADRs: consecuencia de ADR-001. Depende de ADR-002 (BD compartida). Se concreta en ADR-004 (URL del panel como variable de entorno).

ADR-004: Infraestructura y Despliegue

ID	Título	Estado	Fecha
ADR-004	Estrategia de despliegue del sistema en conjunto	Aceptada	01/06/2026

1. Contexto

El sistema tiene dos aplicaciones con requisitos de deployment y runtime distintos, como se definió en ADR-001.

Motivadores:

- 1 Naturaleza opuesta de las cargas de trabajo: la web tiene tráfico impredecible (productores en horario laboral, picos y valles). El chatbot tiene tráfico predecible 24/7.
- 2 Restricciones técnicas opuestas: Next.js está optimizado para serverless. El chatbot requiere proceso persistente para webhook de WhatsApp y estado conversacional.
- 3 Costo: proyecto universitario/startup temprana. El costo debe ser predecible y mínimo.

2. Alternativas consideradas

Alternativa	Ventaja principal en este contexto	Desventaja / Motivo de descarte
Todo en una VPS con Docker Compose	Un solo lugar a operar, costos predecibles, control total.	Un reinicio de la VM tira ambos servicios. Requiere administrar la VM (backups, TLS, monitoring). Pierde integración nativa Next.js ↔ Vercel.
Todo en Kubernetes (clúster gestionado)	Despliegues canarios, escalado horizontal, auto-healing.	Sobreingeniería absoluta para dos servicios y equipo de 6 personas. Curva de aprendizaje demasiado pronunciada. Costo injustificable.
Vercel (web) + Render (chatbot) + Supabase (BD) (ELEGIDA)	Cada servicio en la plataforma óptima para su runtime. Conexión nativa Vercel ↔ Supabase. Cero servidores que administrar.	— (esta fue la elegida)

3. Decisión tomada

Se decide:

- Polizing-Dashboard (panel Next.js) → Vercel: serverless, auto-escala, CDN global, preview deploys por PR.
- chatbot (FastAPI) → Render: instancia containerizada siempre activa (con shutdown cada 15 minutos por ser tier gratuito), URL HTTPS estable para webhook de Meta.
- PostgreSQL → Supabase: managed, backups automáticos, pooler (pgBouncer) integrado.

Fundamentación:

1. Optimización por carga: Vercel escala la web a cero. Render mantiene el chatbot activo 24/7 con costo fijo mínimo. Ideal para patrones de uso opuestos.
2. Restricciones técnicas respetadas: Vercel está hecho para Next.js (Server Components, Edge Cache, preview deploys). Render ejecuta FastAPI como proceso persistente.
3. Sin DevOps requerido: ambas plataformas se configuran desde dashboard + variables de entorno. Deploy automático en cada push a GitHub.

4. Consecuencias

✓ Consecuencias positivas	⚠ Trade-offs / costos
Chatbot siempre disponible con costo fijo mínimo (\$7/mes).	Latencia inter-servicio Vercel → Render (~100ms).
Despliegues y rollbacks independientes por servicio.	Vendor lock-in parcial: migrar de Vercel requiere reingeniería.
Preview deploys del panel en cada PR sin trabajo adicional.	Secretos gestionados en dos plataformas, riesgo de divergencia.

Relación con otros ADRs: concreta ADR-001, ADR-002 y ADR-003 en plataformas de hosting específicas.

ADR-005: Selección del proveedor de mensajería

ID	Título	Estado	Fecha
ADR-005	Selección del proveedor de mensajería como canal con clientes finales	Aceptada	01/06/2026

1. Contexto

El chatbot definido en ADR-001 atiende a clientes finales a través de un canal de mensajería. La interacción incluye texto bidireccional, recepción de imágenes y PDFs (comprobantes de pago, fotos de siniestros, actas policiales) y envío de documentos (tarjeta de circulación en PDF). Para producir esos flujos se necesita elegir un proveedor concreto que conecte el servicio con la red de mensajería que ya usan los clientes.

Motivadores

- 1 Los clientes finales de productores de seguros en Argentina ya tienen WhatsApp instalado y lo usan a diario. Pedirles que instalen una app nueva o usen un canal alternativo (SMS, email) baja drásticamente la adopción. La elección de canal no es libre: es WhatsApp.
- 2 Se necesitan templates aprobados para mensajes iniciados por el negocio (notificaciones de vencimiento), webhooks confiables para entrada (cada mensaje del cliente llega al bot) y APIs de envío programático con soporte de medios (imágenes, PDFs).

2. Alternativas consideradas

Alternativa	Ventaja principal en este contexto	Desventaja / Motivo de descarte
Twilio WhatsApp API (agregador)	SDK maduro, documentación excelente, soporte 24/7, sandbox de pruebas inmediato. Abstrae a Meta.	Costo por mensaje elevado (markup sobre la tarifa de Meta). La capa de abstracción agrega latencia y oculta features que Meta libera primero en su API directa.
Orquestador N8N + Whatsapp API	Privacidad (Self-hosting): Podés alojarlo en tu propia infraestructura, algo vital para no filtrar datos sensibles de los asegurados por servicios de terceros. Conexión ágil: Trae nodos nativos listos para enlazar la API de WhatsApp Cloud y derivar los mensajes directamente al backend, ahorrando programar toda esa capa de autenticación y conexión.	Caos en flujos complejos: Los bots de seguros requieren máquinas de estado (ej: paso 1 pedir DNI, paso 2 patente). Diseñar esto en un editor visual se vuelve un spaghetti inmanejable rápido; es mucho más limpio en código puro. Testing y versionado tosco: No podés correr pruebas unitarias sobre el flujo visual fácilmente,

		y el paso a producción es mucho menos fluido.
Meta WhatsApp Cloud API (directa, elegida)	Acceso oficial directo al canal, tier de mensajes de servicio gratuito muy generoso , números de prueba inmediatos, control total sobre templates y números.	— (esta fue la elegida)

3. Decisión tomada

Se decide: integrar el chatbot directamente con Meta WhatsApp Cloud API (API directa), sin agregadores ni orquestadores intermedios, cubriendo el flujo completo de mensajería:

- **Entrada de mensajes** → webhook HTTPS oficial de Meta: cada mensaje del cliente (texto, imágenes, PDFs) llega al bot de forma confiable.
- **Salida de mensajes** → endpoints de envío de la Cloud API: respuestas de texto y envío de documentos (tarjeta de circulación en PDF) con soporte nativo de medios.
- **Mensajes iniciados por el negocio** → templates aprobados por Meta (notificaciones de vencimiento).
- **Flujos conversacionales** → máquinas de estado implementadas en código sobre FastAPI, versionadas en Git y cubiertas con pruebas unitarias.

Fundamentación:

1. **Canal impuesto por el usuario (motivador 1):** los clientes ya usan WhatsApp a diario. La Cloud API conecta directamente a esa red sin pedirles instalar nada nuevo, maximizando la adopción.
2. **Acceso oficial sin markup (vs Twilio):** ir directo a Meta evita el costo por mensaje del agregador y habilita el tier de servicio gratuito. Las features nuevas llegan primero a la API directa.
3. **Flujos complejos en código, no en visual (vs N8N):** los flujos “paso 1 DNI → paso 2 patente” se modelan limpiamente como máquina de estados en Python, evitando el spaghetti del editor visual.
4. **Soporte nativo de medios (motivador 2):** la Cloud API recibe y envía imágenes y PDFs (comprobantes, siniestros, actas, tarjeta de circulación) sin capas adicionales.

4. Consecuencias

✓ Consecuencias positivas	⚠ Trade-offs / costos
Costo mínimo: sin markup de agregador y con tier de servicio gratuito generoso.	Dependencia directa de un único proveedor (Meta): cambios de política o pricing impactan de lleno.
Control total sobre templates, números y configuración del canal.	Gestión propia del Business Manager y de la aprobación de templates (proceso lento).
Acceso temprano a las features que Meta libera primero en su API directa.	Sin abstracción: hay que implementar a mano la validación del webhook, tokens y reintentos de envío.
Flujos en código → testeables, versionados y mantenibles por todo el equipo.	Vendor lock-in del canal: migrar de proveedor exigiría reescribir la capa de integración.

Relación con otros ADRs: concreta el canal de mensajería del chatbot definido en ADR-001 y se despliega sobre la infraestructura de ADR-004 (webhook HTTPS estable en Render).